

JDBC PreparedStatement & ResultSet Cheat Sheet

This guide covers how to set parameters (PreparedStatement) and retrieve values (ResultSet) for common Java types, with a specific focus on handling **NULL** values correctly to avoid `NullPointerException` or inserting incorrect defaults (like 0 instead of NULL).

1. Setting Parameters (PreparedStatement)

General Rule for Primitives (int, double, boolean): You cannot pass null to `setInt()`, `setDouble()`, etc. You must check if your Wrapper object (Integer, Double) is null. If it is, use `stmt.setNull()`.

Integer / Long / Short

SQL Types: INTEGER, BIGINT, SMALLINT

Integer value = `entity.getAge()`; // or Long, Short

```
if (value == null) {  
    // 1st arg: index, 2nd arg: SQL Type constant  
    stmt.setNull(1, java.sql.Types.INTEGER);  
} else {  
    stmt.setInt(1, value);  
}
```

Double / Float

SQL Types: DOUBLE, FLOAT, REAL

Double value = `entity.getSalary()`;

```
if (value == null) {  
    stmt.setNull(2, java.sql.Types.DOUBLE);  
} else {  
    stmt.setDouble(2, value);  
}
```

String

SQL Types: VARCHAR, CHAR, TEXT

Note: setString handles null natively, but explicit handling is safer if you want specific SQL types.

```
String text = entity.getName();
```

```
// Driver usually handles stmt.setString(3, null) correctly,  
// but this is the explicit way:  
if (text == null) {  
    stmt.setNull(3, java.sql.Types.VARCHAR);  
} else {  
    stmt.setString(3, text);  
}
```

Boolean

SQL Types: BOOLEAN, BIT

```
Boolean isActive = entity.isActive();
```

```
if (isActive == null) {  
    stmt.setNull(4, java.sql.Types.BOOLEAN);  
} else {  
    stmt.setBoolean(4, isActive);  
}
```

BigDecimal (Money/Precision)

SQL Types: DECIMAL, NUMERIC

```
BigDecimal value = entity.getPrice();
```

```
if (value == null) {  
    stmt.setNull(5, java.sql.Types.DECIMAL);  
} else {  
    stmt.setBigDecimal(5, value);  
}
```

Date & Time (Modern Java 8+ java.time)

SQL Types: *DATE, TIMESTAMP, TIME*

*Recommended: Use setObject for Local types, or convert manually.**

A. LocalDate (SQL DATE)

```
LocalDate date = entity.getRegistrationDate();
```

```
if (date == null) {  
    stmt.setNull(6, java.sql.Types.DATE);  
} else {  
    stmt.setDate(6, java.sql.Date.valueOf(date));  
}
```

B. LocalDateTime (SQL TIMESTAMP)

```
LocalDateTime dateTime = entity.getCreatedAt();
```

```
if (dateTime == null) {  
    stmt.setNull(7, java.sql.Types.TIMESTAMP);  
} else {  
    stmt.setTimestamp(7, java.sql.Timestamp.valueOf(dateTime));  
}
```

Enums (Saved as String)

```
UserType type = entity.getType();
```

```
if (type == null) {  
    stmt.setNull(8, java.sql.Types.VARCHAR);  
} else {  
    stmt.setString(8, type.name()); // or type.toString()  
}
```

2. Getting Values (ResultSet)

Critical Warning: Methods like `getInt()`, `getDouble()`, and `getBoolean()` return primitive values (0, 0.0, false) if the database value is **NULL**. To detect if it was actually NULL, you must check `rs.isNull()` immediately after getting the value.

Integer / Long

```
// 1. Get the primitive value
int value = rs.getInt("column_name");

// 2. Check if the LAST read column was null
if (rs.wasNull()) {
    entity.setAge(null); // It was DB NULL, ignore the '0' returned above
} else {
    entity.setAge(value);
}
```

Double / Float

```
double value = rs.getDouble("budget");

if (rs.wasNull()) {
    entity.setBudget(null);
} else {
    entity.setBudget(value);
}
```

Boolean

```
boolean val = rs.getBoolean("is_active");

if (rs.wasNull()) {
    entity.setActive(null);
} else {
    entity.setActive(val);
}
```

String

getString returns null automatically if DB is null, so no extra check is needed.

```
String name = rs.getString("name");
entity.setName(name); // Safe to set directly
```

Date & Time (Java 8 Conversion)

A. LocalDate

```
java.sql.Date sqlDate = rs.getDate("reg_date");

if (sqlDate != null) {
    entity.setRegistrationDate(sqlDate.toLocalDate());
} else {
    entity.setRegistrationDate(null);
}
```

B. LocalDateTime

```
java.sql.Timestamp ts = rs.getTimestamp("created_at");

if (ts != null) {
    entity.setCreatedAt(ts.toLocalDateTime());
} else {
    entity.setCreatedAt(null);
}
```

3. Quick Reference: java.sql.Types

When using `stmt.setNull(index, Type)`, use these constants:

Java Type	SQL Type Constant
Integer	<code>java.sql.Types.INTEGER</code>
Long	<code>java.sql.Types.BIGINT</code>
Double	<code>java.sql.Types.DOUBLE</code>
String	<code>java.sql.Types.VARCHAR</code>
Boolean	<code>java.sql.Types.BOOLEAN</code>
LocalDate	<code>java.sql.Types.DATE</code>
LocalDateTime	<code>java.sql.Types.TIMESTAMP</code>

BigDecimal	java.sql.Types.DECIMAL
Float	java.sql.Types.FLOAT

4. Generic Helper Method (Optional)

If you have many fields and want to write less code, you can use a helper method in your Repository:

```
private void setIntOrNull(PreparedStatement stmt, int index, Integer value) throws
SQLException {
    if (value == null) {
        stmt.setNull(index, java.sql.Types.INTEGER);
    } else {
        stmt.setInt(index, value);
    }
}
```

```
// Usage:
setIntOrNull(stmt, 1, entity.getId());
```